CSE-1003 • MODULE 2 • CLASS 3

Repeat until done

Riya's friends keep ordering. Today the program learns to repeat, to count, to draw patterns, and to know when to stop.

Basic Programming and Control Statements • Dr Sachin D N • IIIT Bhopal

TODAY'S SYLLABUS TOPICS

Three topics, one canteen that never closes

1

Iterative statements

while, for, do-while

Which loop when?

An unsolved puzzle

2

Nested loops

A loop inside a loop

Cricket overs, cinema seats

Star patterns

3

break and continue

break: stop early

continue: skip one round

Fix the broken table

The last part of Module 2. After today you can write programs that run any number of times.

BEFORE WE START

Print “Thank you, Riya!” 1000 times. How?

Without a loop

```
printf("Thank you, Riya!\n");
printf("Thank you, Riya!\n");
printf("Thank you, Riya!\n");
printf("Thank you, Riya!\n");
printf("Thank you, Riya!\n");
printf("Thank you, Riya!\n");
… 994 more lines …
```

1000 lines

With a loop

```
for (i = 1; i ⩽ 1000; i++) {
    printf("Thank you, Riya!\n");
}
```

3 lines

Change 1000 to 1,00,000 and it's still 3 lines.

TODAY'S GOAL

A canteen counter that keeps taking orders

It **repeats** (loop), **skips** a wrong code (continue), **stops** on 0 (break), and draws a line (another loop).

```
1 Chai 2 Samosa 3 Maggi, 0 to finish
Item code: 1
Quantity: 2
Item code: 5
No such item, try again
Item code: 2
Quantity: 1
Item code: 3
Quantity: 1
Item code: 0
-----------------------
Items: 4 Total: Rs 75
```

TOPIC 1 OF 3 • ITERATIVE STATEMENTS

What is a loop? Stirring chai until it boils



You don't stir once and walk away. You **repeat**: check if it's boiling, stir, check again… until it's done. Every loop has the same four parts.

1 • Start

Pour milk and tea

```
i = 1
```

2 • Check

Is it boiling yet?

```
i ⩽ 5
```

3 • Do

Stir once

```
printf(...)
```

4 • Change

The heat rises

```
i++
```

Forget part 4? The heat never rises, the check never fails, and you stir forever: an **infinite loop**.

TOPIC 1 OF 3 • ITERATIVE STATEMENTS

for or while? Do you know the count, or the stopping point?



for

I know HOW MANY times

Coach says: do **10 push-ups**. You know the count before you start.

```
for (i = 1; i ⩽ 10; i++)
    printf("Push-up %d\n", i);
```

Use it for:

- Table of 7, from 1 to 10
- Marks of all 60 students



while

I know WHEN to stop

Stir the chai **until it boils**. Nobody knows how many stirs that takes.

```
while (!boiling)
    stir();
```

Use it for:

- Take orders until the customer says 0
- Ask for the PIN until it is correct

TOPIC 1 OF 3 • ITERATIVE STATEMENTS

while: count down until liftoff



Rocket launch countdown, like ISRO's **5… 4… 3… 2… 1… Liftoff!**

```
1  int count = 5;
2  while (count > 0) {
3      printf("%d ", count);
4      count--;
5  }
6  printf("Liftoff!");
```

- Start: count = 5
- Check: count > 0?
- Do: print the number
- Change: count goes down by 1

#	count	> 0?	Prints	What happens
1	5	Yes	5	Check passes: print 5, then count becomes 4.
2	4	Yes	4	Back to the check.
3	3	Yes	3	
4	2	Yes	2	
5	1	Yes	1	Last number printed.
6	0	No	Liftoff!	Check fails: leave the loop, run line 6.

The check runs 6 times; the body runs 5 times.

TOPIC 1 OF 3 • ITERATIVE STATEMENTS

for: when you know how many rounds

```
for (i = 1; i ⩽ 5; i++) { … }
```

`i = 1`
Start: runs once

`i ⩽ 5`
Check: before every round

`i++`
Change: after every round

Real world

Pour chai for **5 friends**. You know the count before you start: one cup, two, … five.

The same loop with while

```
i = 1;
while (i ⩽ 5) {
    …
    i++;
}
```

TOPIC 1 OF 3 • ITERATIVE STATEMENTS

for, step by step: the chai price board



The canteen needs a price board for 1 to 5 cups of chai at ₹10 each.

```c
for (cups = 1; cups ⩽ 5; cups++) {
    printf("%d cups = Rs %d\n",
           cups, cups * 10);
}
```

- **1 • Start**: cups = 1 (only once)
- **2 • Check**: cups <= 5?
- **3 • Do**: print one line
- **4 • Change**: cups++, then back to 2

#	cups	<= 5?	Prints	What happens
1	1	Yes	1 cups = Rs 10	Start sets cups = 1. Check passes.
2	2	Yes	2 cups = Rs 20	cups++ made it 2. Check again.
3	3	Yes	3 cups = Rs 30	
4	4	Yes	4 cups = Rs 40	
5	5	Yes	5 cups = Rs 50	Last line printed.
6	6	No	-	6 <= 5 is false: the loop ends.

Start runs once. Check runs 6 times. Do and change run 5 times.

TOPIC 1 OF 3 • ITERATIVE STATEMENTS

do-while: do it first, check afterwards

The ATM can't check a PIN it hasn't asked for. So it **asks first**, then checks.

```
1  int pin;
2  do {
3      printf("Enter PIN: ");
4      scanf("%d", &pin);
5  } while (pin ≠ 1234);
6  printf("Welcome!");
```

- **1 • Do first**: ask for the PIN (always, at least once)
- **2 • Check after**: pin != 1234? If yes, go back to 1
- **Watch out**: do-while ends with a semicolon ;

#	Round	Riya types	!= 1234?	What happens
1	1	1111	Yes	Asked once without any check. Wrong, so repeat.
2	2	4321	Yes	Still wrong: ask again.
3	3	1234	No	Correct: the check fails, leave the loop, print Welcome!

while checks first (may run 0 times). do-while checks after (always runs at least once).

TOPIC 1 OF 3 • ITERATIVE STATEMENTS • QUICK GAME

Which loop would you use? Shout it out!

Know the count? `for` → Must run at least once? `do-while` → Otherwise: stop on a condition `while`

1

Print the table of 7, from 7 × 1 to 7 × 10

for: the count is 10

2

Take orders until the customer types 0

while: stop on 0

3

Show the canteen menu, then read the choice

do-while: menu shows first

4

Mark attendance for all 60 students

for: the count is 60

5

Play a game until lives become 0

while: stop when lives = 0

6

After each game ask: Play again? (y/n)

do-while: ask at least once

CURIOUS CORNER • ITERATIVE STATEMENTS

A puzzle nobody has solved, in one while loop

The rules (repeat while n is not 1)

Even: **n = n / 2**

Odd: **n = 3 × n + 1**

```
while (n ≠ 1) {
    if (n % 2 == 0)
        n = n / 2;
    else
        n = 3 * n + 1;
    steps++;
}
```

Scan and try it yourself

Start with 27, then your roll number.

onlinegdb.com/zF1-YZZV3

Start with 6. Predict the next number before each click:

6	3	10	5	16
8	4	2	1	

27 takes 111 steps and climbs as high as **9232** before it falls to 1.

Does **every** number reach 1? Proposed in 1937 by Lothar Collatz. Computers have checked enormous numbers, but nobody has **proved** it.

Try your roll number: onlinegdb.com/zF1-YZZV3

TOPIC 1 OF 3 • ITERATIVE STATEMENTS • PREDICT

How many times? What prints?

1
```
for (i = 0; i < 5; i++) printf("*");
```

***** (5 times)

2
```
int i = 10; while (i < 5) { printf("Hi"); }
```

Nothing: 0 times

3
```
int i = 10; do { printf("Hi"); } while (i < 5);
```

Hi: once

4
```
for (i = 1; i ⩽ 10; i += 3) printf("%d ", i);
```

1 4 7 10

TOPIC 1 OF 3 • ITERATIVE STATEMENTS • MISSING LINE CHALLENGE

One line is missing. Which line, and where?

```c
/* One line is missing. Find it and write it. */
#include <stdio.h>

int main(void) {
    int n, i = 1, sum = 0;

    printf("Add 1 to n. n? ");
    scanf("%d", &n);

    while (i ⩽ n) {
        sum = sum + i;
        /* ??? missing line ??? */
    }

    printf("Sum = %d\n", sum);
    return 0;
}
```

Run it with n = 5. Instead of 15 it **never finishes**. Write the missing line and say where it goes.

Correct output

```
Add 1 to n. n? 5
Sum = 15
```

Scan or open:
onlinegdb.com/XTp09_nnB

Answer: i++; after line 11, inside the loop. Without it, i stays 1, **i <= n** is always true, and the loop never ends: an infinite loop.

TOPIC 2 OF 3 • NESTED LOOPS

Nested loops: a loop inside a loop, like overs and balls

A T20 innings

Over 1 1 2 3 4 5 6

Over 2 1 2 3 4 5 6

Over 3 1 2 3 4 5 6

… up to over 20

For **each over**, bowl **6 balls**. 20 × 6 = **120 balls**.

```
for (over = 1; over ⩽ 20; over++) {
    for (ball = 1; ball ⩽ 6; ball++) {
        printf("%d.%d ", over, ball);
    }
}
```

Outer loop: the overs. **Inner loop:** the balls.

The inner loop runs **completely** for every single round of the outer loop. A clock works the same way: 60 minutes for every hour.

TOPIC 2 OF 3 • NESTED LOOPS

Inside a nested loop, step by step

```
1  for (over = 1; over ⩽ 2; over++) {
2      for (ball = 1; ball ⩽ 3; ball++) {
3          printf("%d.%d ", over, ball);
4      }
5  }
```

A short game: 2 overs of 3 balls.

#	over	ball	Prints	What happens
1	1	1	1.1	Outer loop starts over 1.
2	1	2	1.2	Inner loop moves on.
3	1	3	1.3	Inner loop finishes.
4	2	1	2.1	Outer moves to over 2; inner restarts at 1.
5	2	2	2.2	
6	2	3	2.3	Both loops finish.

Total rounds = outer × inner = 2 × 3 = 6.

TOPIC 2 OF 3 • NESTED LOOPS

Real world: a cinema seat map

```c
/* Cinema seat chart: rows A to E, seats 1 to 6 */
#include <stdio.h>

int main(void) {
    char row;
    int seat;

    for (row = 'A'; row ⩽ 'E'; row++) {
        for (seat = 1; seat ⩽ 6; seat++) {
            printf("%c%d ", row, seat);
        }
        printf("\n");
    }
    return 0;
}
```

SCREEN

```
A1 A2 A3 A4 A5 A6
B1 B2 B3 B4 B5 B6
C1 C2 C3 C4 C5 C6
D1 D2 D3 D4 D5 D6
E1 E2 E3 E4 E5 E6
```

Outer loop: rows A to E. Inner loop: seats 1 to 6. Line 12 starts each new row.

TOPIC 2 OF 3 • NESTED LOOPS

The star triangle, row by row

```
1  for (i = 1; i ⩽ 4; i++) {
2      for (j = 1; j ⩽ i; j++) {
3          printf("* ");
4      }
5      printf("\n");
6  }
```

The inner loop runs **i** times, so each row has one more star. Exams love this one.

#	i	j runs	Row printed	What happens
1	1	1 time	*	
2	2	2 times	* *	
3	3	3 times	* * *	
4	4	4 times	* * * *	Outer loop ends after i = 4.

Change j <= i to j <= 5 – i and the triangle flips.

TOPIC 2 OF 3 • NESTED LOOPS • PREDICT THE PICTURE

What does it draw?

1

```
for i 1 to 3: { for j 1 to 2: print "#"; newline }
```

```
##
##
##
```

2

```
for i 1 to 3: { for j 1 to i: print j; newline }
```

```
1
12
123
```

3

```
for i 1 to 3: { for j 1 to 3: print (i == j) ? 1 : 0;
newline }
```

```
100
010
001
```

TOPIC 2 OF 3 • NESTED LOOPS • MISSING LINE CHALLENGE

All the stars on one line? Something's missing

```c
/* One line is missing. Find it and write it. */
#include <stdio.h>

int main(void) {
    int i, j;

    for (i = 1; i <= 4; i++) {
        for (j = 1; j <= i; j++) {
            printf("* ");
        }
        /* ??? missing line ??? */
    }
    return 0;
}
```

Answer: printf("\n"); on line 11, after the inner loop and inside the outer loop. It ends each row.

It prints * * * * * * * * * * on **one** line. Write the missing line.

Correct output

```
*
* *
* * *
* * * *
```



Scan or open:
onlinegdb.com/m04omS1u2

TOPIC 3 OF 3 • BREAK AND CONTINUE

break stops the whole loop. continue skips just one round.

break;

Stop the loop completely

Finding your friend in the queue: once found, you **stop** searching. Here, round 3 stops it.

1	2	3	4	5
prints	prints	break!	never runs	never runs

```
for (i = 1; i ⩽ 5; i++) {
    if (i == 3) break;
    printf("%d ", i);
}
```

```
Output: 1 2
```

continue;

Skip this round, keep going

A song you don't like: you **skip** it and the playlist plays on. Here, round 3 is skipped.

1	2	3	4	5
prints	prints	skipped	prints	prints

```
for (i = 1; i ⩽ 5; i++) {
    if (i == 3) continue;
    printf("%d ", i);
}
```

```
Output: 1 2 4 5
```

TOPIC 3 OF 3 • BREAK AND CONTINUE

break in action: an ATM with 3 tries

```
 1  for (attempt = 1; attempt ⩽ 3;
attempt++) {
 2      printf("PIN: ");
 3      scanf("%d", &pin);
 4      if (pin == 1234) {
 5          printf("Welcome!");
 6          break;
 7      }
 8      printf("Wrong PIN\n");
 9  }
```

Riya types **1111**, then **1234**.

#	attempt	pin	Prints	What happens
1	1	1111	`Wrong PIN`	Not 1234: no break, next round.
2	2	1234	`Welcome!`	Correct: break runs.
3	3	–	–	Never happens: break already left the loop.

break jumps straight out, even if rounds are left.

TOPIC 3 OF 3 • BREAK AND CONTINUE

continue in action: skip the even seats

```
1  for (i = 1; i ⩽ 6; i++) {
2      if (i % 2 == 0)
3          continue;
4      printf("%d ", i);
5  }
```

Exam seating: students sit only in **odd** numbered seats, so the program skips the even ones.

#	i	Even?	Prints	What happens
1	1	No	1	
2	2	Yes	–	continue: skip printf, go to i++.
3	3	No	3	
4	4	Yes	–	Skipped.
5	5	No	5	
6	6	Yes	–	Skipped. Loop ends.

Output: 1 3 5. continue skips the rest of ONE round.

TOPIC 3 OF 3 • BREAK AND CONTINUE • PREDICT

What prints?

1

```
for (i = 1; i ⩽ 10; i++) { if (i == 4) break; printf("%d
", i); }
```

123

2

```
for (i = 1; i ⩽ 6; i++) { if (i == 4) continue;
printf("%d ", i); }
```

12356

3

```
n = 1; while (1) { n = n * 2; if (n > 100) break; }
printf("%d", n);
```

128

TOPIC 3 OF 3 • BREAK AND CONTINUE • FIX THE BROKEN CODE

Table of 7: why does it print only one line?

```c
/* Table of 7, from 7 x 1 to 7 x 10: find and fix 3 bugs */
#include <stdio.h>

int main(void) {
    int i;

    for (i = 1; i < 10; i++);
    {
        printf("7 x %d = %d\n", i, 7 + i);
    }
    return 0;
}
```

It prints only this:

```
7 x 10 = 17
```

It should print 7 × 1 to 7 × 10. Find the **3 bugs**, fix them, run it.

Line 7: ; after the for

Loop body runs only once, after the loop

Line 7: < 10 should be <= 10

7 × 10 would be missed

Line 9: 7 + i should be 7 * i

Logical error: wrong maths

Scan to fix it

onlinegdb.com/aAWDAk8fH

PUTTING IT ALL TOGETHER

Our use case: Riya at the canteen counter

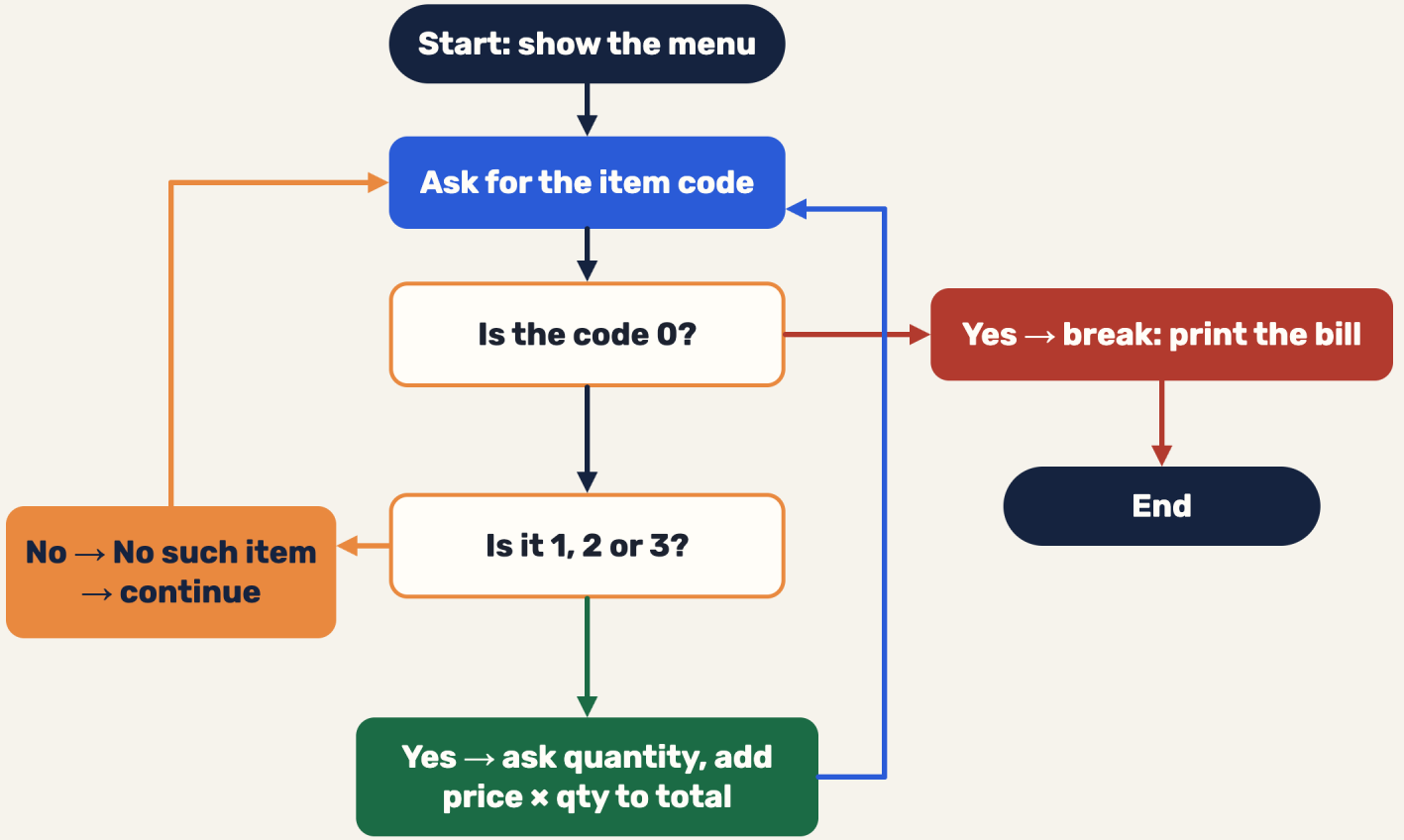
What Riya types

Item code • quantity

```
1 · qty 2   + Rs 20
5           No such item, ask again
2 · qty 1   + Rs 15
3 · qty 1   + Rs 40
0           Done: stop taking orders

Items: 4 Total: Rs 75
```

Blue = the loop repeats, **orange** = continue, **red** = break.



ALL 3 TOPICS TOGETHER

The canteen counter that keeps taking orders

7	**1 • while (1): repeat until a break**
10–11	**3 • break: 0 ends the orders**
12–14	**3 • continue: skip a wrong code**
21	**Keep a running total**
25	**1 • for: draw the line**

The if-else for prices is from Class 2.

```c
#include <stdio.h>

int main(void) {
    int code, qty, price, total = 0, items = 0, i;

    printf("1 Chai 2 Samosa 3 Maggi, 0 to finish\n");
    while (1) {
        printf("Item code: ");
        scanf("%d", &code);
        if (code == 0)
            break;
        if (code < 1 || code > 3) {
            printf("No such item, try again\n");
            continue;
        }
        printf("Quantity: ");
        scanf("%d", &qty);
        if (code == 1) price = 10;
        else if (code == 2) price = 15;
        else price = 40;
        total += price * qty;
        items += qty;
    }

    for (i = 0; i < 24; i++) printf("-");
    printf("\nItems: %d Total: Rs %d\n", items, total);
    return 0;
}
```

RUN IT YOURSELF

Run the counter, then we'll check your predictions

`canteen3.c`

1. Open the link or scan the QR code
2. Type the input in the input box: **1 2 5 2 1 3 1 0**
3. Press **Run**
4. Check: does it print **Items: 4, Total: Rs 75**?



onlinegdb.com/fEd5M0-HI

PREDICT BEFORE YOU RUN

What will the counter do?

1 Codes: 1 (qty 2), 9, 3 (qty 1), 0

No such item for 9; Items: 3, Total: Rs 60

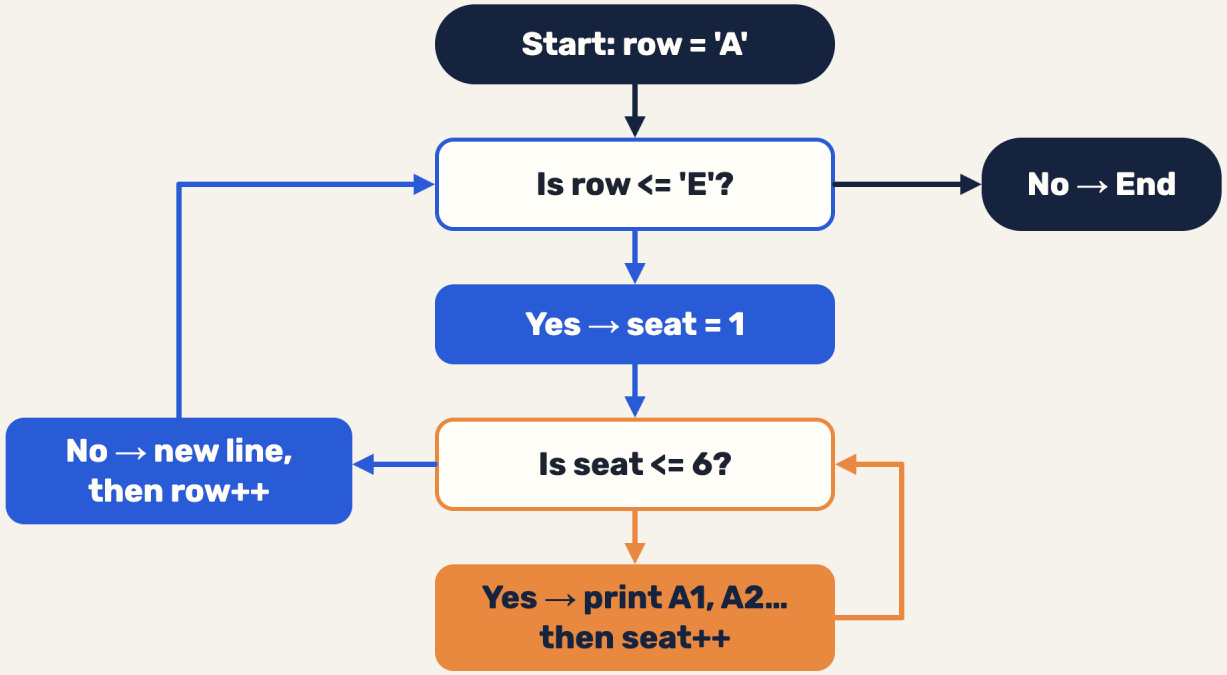
2 Riya types 0 straight away

Items: 0, Total: Rs 0

3 What if line 'continue;' were deleted?

9 would ask for a quantity and be charged as Maggi

Cinema seats: how the two loops take turns



SCREEN

```
A1 A2 A3 A4 A5 A6
B1 B2 B3 B4 B5 B6
C1 C2 C3 C4 C5 C6
D1 D2 D3 D4 D5 D6
E1 E2 E3 E4 E5 E6
```

onlinegdb.com/_zOm6Xm2Y

Orange: inner loop (seats)

Blue: outer loop (rows)

Try: 8 rows of 10 seats.

YOUR TURN · MISSING LINES

Write the 3 missing lines

```c
/* 3 lines are missing. Write each one. */
#include <stdio.h>

int main(void) {
    int code, qty, price, total = 0;

    while (1) {
        printf("Item code (0 to finish): ");
        scanf("%d", &code);
        if (code == 0)
            /* ??? missing line 1: stop taking orders ??? */
        if (code < 1 || code > 3) {
            printf("No such item\n");
            /* ??? missing line 2: ask for the next code ??? */
        }
        printf("Quantity: ");
        scanf("%d", &qty);
        if (code == 1) price = 10;
        else if (code == 2) price = 15;
        else price = 40;
        /* ??? missing line 3: add this item to total ??? */
    }
    printf("Total: Rs %d\n", total);
    return 0;
}
```

Each comment says **what** the line must do.
You decide **how**.

Test: 1 2, 5, 3 1, 0

```
No such item
Total: Rs 60
```

onlinegdb.com/oG63uc1EI

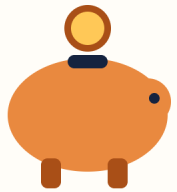
Line 11 `break;`

Line 14 `continue;`

Line 21 `total += price * qty;`

YOUR TURN • TASK 1 OF 3

Savings goal



Riya saves the same amount every day for a ₹1000 phone. How many days until she has enough?

while **unknown rounds**

Pseudocode

```
READ goal, perday
saved ← 0, days ← 0
WHILE saved < goal
    saved ← saved + perday
    days ← days + 1
END WHILE
PRINT days, saved
```

Sample runs

```
Goal and saving per day?
1000 50
Days needed: 20 (saved
Rs 1000)

Goal and saving per day?
1000 70
Days needed: 15 (saved
Rs 1050)
```

Check yourself

- ✓ Why while, not for?
- ✓ Test 70 a day: the last day overshoots
- ✓ What if perday is 0? (careful!)

Now: Write it in C in OnlineGDB. Test every sample.

YOUR TURN • TASK 2 OF 3

Sum of the digits of your roll number



Read a number and add up its digits. 2026 gives 2 + 0 + 2 + 6 = 10.

while **% 10 and / 10**

Pseudocode

```
READ n
sum ← 0
WHILE n > 0
    sum ← sum + (n MOD 10)
    n ← n / 10
END WHILE
PRINT sum
```

Sample runs

```
Number? 2026
Sum of digits = 10
```

Check yourself

- ✓ % 10 gives the last digit
- ✓ / 10 removes the last digit
- ✓ Try your own roll number

Now: Write it in C in OnlineGDB. Test every sample.

YOUR TURN • TASK 3 OF 3

Number pyramid



Read the number of rows and print this pyramid. The star triangle used the same idea.

nested loops

inner limit = row

Pseudocode

```
READ rows
FOR i ← 1 TO rows
    FOR j ← 1 TO i
        PRINT j
    END FOR
    PRINT new line
END FOR
```

Sample runs

```
Rows? 4
1
1 2
1 2 3
1 2 3 4
```

Check yourself

- ✓ Start from the star triangle
- ✓ Print j instead of a star
- ✓ Bonus: print i instead. What changes?

Now: Write it in C in OnlineGDB. Test every sample.

CURIOUS CORNER • LOOPS AND DATA TYPES

The chessboard puzzle: a story about doubling

The king

"You invented chess! Ask me for any reward."

The inventor

"Only rice. 1 grain on the first square, then **double** it on every next square."

The king laughs

"Just rice? Granted! Bring him a small bag."

Square 1	Square 2	Square 3	Square 4	Square 5	Square 6	Square 7	Square 8
1	2	4	8	16	32	64	128

Your guess? The first row ends at just 128 grains. How many grains on square 64? A thousand? A million? More?

CURIOUS CORNER · LOOPS AND DATA TYPES

The loop that counts the rice

```
unsigned long long grains = 1, total = 0;
for (square = 1; square ⩽ 64; square++) {
    total = total + grains;
    grains = grains * 2;
}
```

One small loop, 64 rounds. Each round adds the grains to the total and **doubles** them for the next square.

Run it yourself

Then try int instead of unsigned long long.

onlinegdb.com/DVDy76PPW



Square 10 512

A small handful.



Square 32 2,147,483,648

Truckloads. Too big for an int, so we use unsigned long long.



Square 64 alone 9,223,372,036,854,775,808

Roughly hundreds of years of the whole world's rice harvest.



All 64 added 18,446,744,073,709,551,615

Almost double square 64: each square holds more than all earlier squares combined. Also the biggest number an unsigned long long can hold.

TODAY IN ONE SLIDE

Repeat, nest, stop and skip

1

Loops

for when you know the count, while when you wait for a condition, do-while when it must run once. Every loop needs start, check, do and change.

2

Nested loops

The inner loop runs completely for every outer round: overs and balls, rows and seats, star patterns.

3

break and continue

break leaves the loop at once. continue skips the rest of one round and carries on.

That completes Module 2: from your first printf to a counter that runs until the last customer leaves.

COMING NEXT • MODULE 3

Riya's program is getting long. What if we could name a block of code and reuse it anywhere?

Module 3: functions and recursion.

Before next class: finish one task and fix the broken table.